

Compte rendu de stage

**Conception de la version embarquée
d'un ergocycle à bras avec adaptation
temps réel basée sur les EMG**

Laboratoire S2M - Université de Montréal

Yasmine Salmouni

Pr. Mickael Begon

Aout 2025



Table des matières

1	Introduction	3
2	Prise en main du projet existant	4
2.1	Description générale du dispositif existant	4
2.2	Développement de la version embarqué del'ergocycle S2M	4
3	Développement du matériel (hardware) de la version embarquée de l'ergo- cycle	6
3.1	Composants électroniques du projet	6
3.1.1	Moteur Tongsheng TSDZ2, 48V, 500 W	6
3.1.2	Contrôleur Flipsky FSESC 4.12 50A	6
3.1.3	Ecran	7
3.1.4	Anti spark	8
3.1.5	Odrive Regen Clamp	9
3.1.6	Batterie Li-ion 48V – 10Ah	10
3.1.7	Photos de la version embarquée finalisée de l'ergocycle	11
4	Développement du logiciel (software) de la version embarquée de l'ergocycle	13
4.1	Choix initiaux	13
4.1.1	Passage de Python orienté objet à C++	13
4.1.2	Justification du choix du microcontrôleur STM32F407G-DISC1	13
4.1.3	Environnement de développement (STM32 Cube IDE)	14
4.2	Architecture logicielle modulaire du projet	18
4.3	Méthode de calibration de la constante de couple Kt du moteur dans le projet	19
4.4	Gestion de la batterie	20
4.5	Installation du firmware du contrôleur VESC	20
4.6	Conception de l'interface utilisateur graphique (GUI) de l'écran	23
5	Tests du système	27
5.1	Problème de liaison entre le pédalier le moteur	27
5.2	Latences	28
5.2.1	Installation de l'OS FreeRTOS	28
6	Exploitation des signaux électromyographiques (EMG) pour adapter l'assis- tance de l'ergocycle embarqué et prolonger l'effort musculaire de l'utilisateur	29
6.1	Première étape : développement d'un modèle de prédiction du temps restant avant l'épuisement musculaire.	30
6.1.1	Architecture de la base de données	31
6.1.2	Prtraitement des données	31
6.1.3	Développement et entraînementdu modèle	33
6.1.4	Résultats	33
6.2	Deuxième étape : Acquisition des signaux EMG	35
6.3	Test du système	36



7 Conclusion	37
7.1 Perspectives d'amélioration	37
Liste des figures	38



1 Introduction

Dans le cadre de ma première année de Master en systèmes embarqués et microélectronique, j'effectue actuellement un stage au sein du laboratoire S2M à l'Université de Montréal. Mon sujet de stage porte sur le développement d'une version embarquée d'un ergocycle à bras instrumentés. L'objectif de ce projet est de concevoir un dispositif autonome et compact, destiné à assister les personnes à mobilité réduite dans leur processus de rééducation ou d'entraînement sportif adapté.

Une première version de cet ergocycle existe déjà au laboratoire ; cependant, celle-ci n'est pas embarquée, ce qui limite fortement son utilisation hors d'un environnement contrôlé. Mon rôle consiste donc à adapter et repenser l'ensemble de cette solution existante en proposant une implémentation matérielle et logicielle totalement embarquée. Pour ce faire, j'ai dû prendre en main le projet initial, comprendre la logique sur laquelle il repose, et déterminer les évolutions nécessaires pour répondre à l'objectif fixé.

Dans ce rapport d'étape, je présenterai dans un premier temps la démarche que j'ai suivie afin de comprendre le projet existant. Je reviendrai ensuite en détail sur les travaux réalisés, aussi bien sur le plan logiciel que matériel. Enfin, je présenterai les défis techniques que je rencontre actuellement, ainsi que les prochaines étapes prévues.



2 Prise en main du projet existant

2.1 Description générale du dispositif existant

L'Ergocycle à bras instrumenté développé au laboratoire S2M est un dispositif d'entraînement pour les bras. Il s'agit d'un système motorisé et contrôlé par ordinateur qui permet de faire travailler un utilisateur soit en concentrique (l'utilisateur fait tourner les manivelles. Le moteur freine légèrement pour créer une résistance, mais ne tourne pas tout seul.) soit en excentrique (le moteur fait tourner les manivelles.). L'ergocycle est composé d'un moteur électrique relié à un pédalier manuel et piloté par un contrôleur de moteur, le tout supervisé par un raspberry pi.

Quatre modes de contrôle principaux sont proposés à l'utilisateur, avec une disponibilité qui dépend du mode d'entraînement (concentrique ou excentrique) :

- Mode Torque : dans ce mode, on impose un couple résistant constant que le moteur va appliquer aux manivelles, indépendamment de la vitesse de rotation.
- Mode Cadence : dans ce mode, le moteur cherche à maintenir une cadence de pédalage fixe quel que soit le couple appliqué par l'utilisateur.
- Mode Power : dans ce mode, le but est de contrôler la puissance que fournit (mode concentrique) ou absorbe (mode excentrique) l'utilisateur.
- Mode Linear : La résistance (couple) appliquée par le moteur augmente proportionnellement à la vitesse de pédalage.

2.2 Développement de la version embarqué del'ergocycle S2M

Pour développer la version embarquée de cet ergocycle, j'ai repensé son fonctionnement afin de le rendre adapté à un usage en extérieur. La version finale repose sur un principe simple, avec deux directions de fonctionnement : **FORWARD** (vers l'avant) et **REVERSE** (vers l'arrière), ainsi que cinq modes principaux de pilotage :

- **Mode CADENCE :**
Objectif : Maintenir une vitesse de rotation (RPM : tours de pédalier par minute) cible. L'utilisateur fournit une consigne de cadence (en tr/min). L'utilisateur pédale librement, et le moteur s'adapte à sa cadence pour fournir l'assistance nécessaire et atteindre la cadence cible définie.
- **Mode TORQUE :**
Objectif : Appliquer un couple au pédalier (en Nm).
L'utilisateur fournit une consigne de couple positif (en Nm). Le moteur applique le couple demandé, indépendamment de la vitesse.
 - En FORWARD : le couple est appliqué dans le sens du mouvement, pour assister l'utilisateur.
 - En REVERSE : le couple est appliqué en opposition au mouvement, pour créer un freinage ou une résistance.
- **Mode POWER_CONCENTRIC :**
Objectif : Délivrer une puissance mécanique positive (effort concentrique).



L'utilisateur pédale librement, et le moteur ajuste le couple appliqué pour délivrer une puissance cible (en watts), selon la relation $P = \tau \cdot \omega$, où τ est le couple appliqué par le moteur et ω la vitesse angulaire liée à la cadence C (en tr/min) par la formule $\omega = \frac{2\pi C}{60}$. Ce mode fonctionne uniquement dans le sens Forward.

— **Mode POWER_ECCENTRIC :**

Objectif : Délivrer une puissance mécanique négative (effort excentrique).

Comme en mode POWER_CONCENTRIC, l'utilisateur pédale librement et entre une valeur cible de puissance (valeur absolue en watts). La différence réside dans le sens du couple appliqué par le moteur : ici, il agit comme une résistance au mouvement. Le moteur adapte ce couple résistant pour atteindre une puissance cible négative, selon la relation $P = -|\tau| \cdot \omega$, avec $\omega = \frac{2\pi C}{60}$. Ce mode fonctionne uniquement dans le sens Forward.

— **Mode LINEAR :**

Objectif : Appliquer un couple proportionnel à la cadence.

L'utilisateur fournit un gain linéaire, et le moteur applique un couple selon la relation suivante :

$$\text{couple} = \text{gain} \times \text{cadence}$$

- En FORWARD : le couple appliqué assiste le mouvement.
- En REVERSE : le couple appliqué crée une résistance.

3 Développement du matériel (hardware) de la version embarquée de l'ergocycle

3.1 Composants électroniques du projet

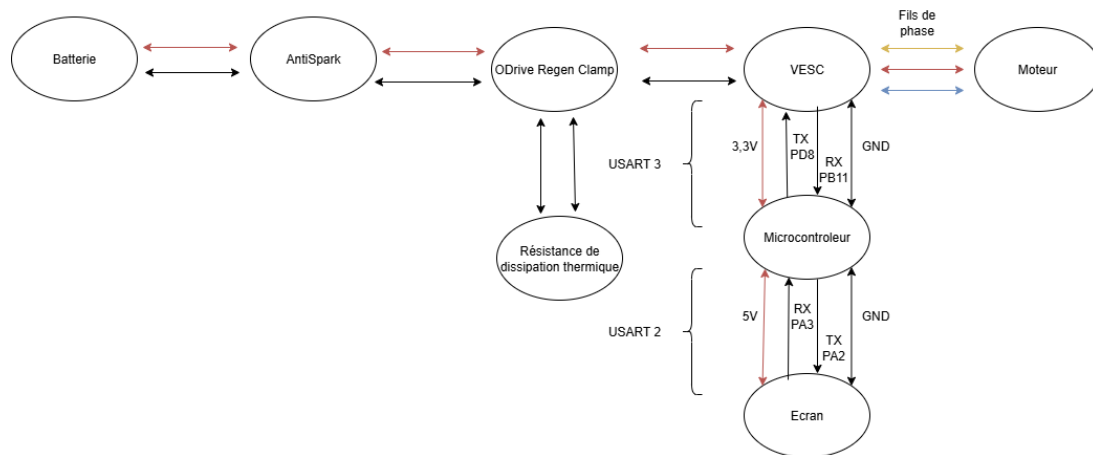


FIGURE 1 – Architecture Hardware du projet

3.1.1 Moteur Tongsheng TSDZ2, 48V, 500 W

Le moteur est équipé d'un contrôleur intégré. Pour pouvoir le piloter avec un contrôleur externe, il est nécessaire de retirer ce contrôleur interne. Comme les circuits ne sont pas conçus pour être facilement démontés, il faudra dévisser fermement, retirer les fixations (colle, résine, etc.) et extraire le contrôleur même si cela demande un peu de force.



FIGURE 2 – Contrôleur intégré au moteur

Les trois fils de phase, désormais déconnectés du contrôleur interne, doivent être raccordés au contrôleur externe pour permettre le pilotage du moteur.

3.1.2 Contrôleur Flipsky FSESC 4.12 50A

Le contrôleur FSESC 4.12 50A possède trois fils de phase, mais contrairement à ceux du moteur, ils ne sont pas différenciés par des couleurs. Par conséquent, le raccordement se fait sans



ordre particulier : il suffit de les connecter de manière aléatoire. Si le moteur tourne dans le mauvais sens, il suffit d'inverser deux fils pour corriger la direction de rotation.

Les fils d'alimentation du contrôleur doivent être raccordés à la batterie.

Les fils Hall ne sont pas raccordés au moteur, le contrôleur fonctionne ainsi en mode capteur sensorless.

Le contrôleur communique via liaison série avec le microcontrôleur, les pins RX, TX, 3,3V et GND du contrôleur sont donc connectés aux pins correspondants du microcontrôleur (Voir Figure 1)



FIGURE 3 – Contrôleur Flipsky FSESC 4.12 50A

3.1.3 Ecran

J'ai utilisé un écran tactile HMI Nextion intelligent de 4,3 pouces afin d'afficher l'interface graphique (GUI) de mon ergocycle embarqué, permettant ainsi une interaction entre l'utilisateur et le système.



3 Développement du matériel (hardware) de la version embarquée de l'ergocycle



FIGURE 4 – Ecran Nextion intelligent de 4,3 pouces

3.1.4 Anti spark



FIGURE 5 – Anti spark Switch Smart

Quand on branche la batterie au contrôleur, les condensateurs à l'entrée du contrôleur se chargent d'un coup, ce qui produit une étincelle, un "spark", ceci use prématurément les composants et abîme les connecteurs.

Un antispark est un système qui permet de retarder le courant au moment du branchement. Il évite donc l'étincelle. D'abord, il laisse passer un petit courant pour charger doucement les condensateurs. Ensuite, il ferme complètement le circuit pour que le moteur fonctionne normalement, sans étincelle.

L'entrée de l'antispark est branchée à la sortie de la batterie et sa sortie à l'entrée du contrôleur.



Le bouton sur l'anti spark permet d'allumer et éteindre le système sans avoir à éteindre et allumer la batterie à chaque usage. Pour éviter les fuites de courant, je recommande d'éteindre complètement la batterie, et non simplement l'antispark, une fois que l'ergocycle n'est plus utilisé.

3.1.5 Odrive Regen Clamp

Lorsqu'on freine avec un vélo électrique, le moteur peut fonctionner en mode génératrice. Cela signifie que le moteur convertit l'énergie mécanique (le mouvement des pédales) en énergie électrique, qu'il renvoie dans le circuit électrique. Cette énergie produite par le freinage doit nécessairement aller quelque part. En situation normale, cette énergie retourne vers la batterie : c'est ce qu'on appelle le freinage régénératif. Toutefois, lorsque la batterie est déjà chargée à pleine capacité ou presque, elle ne peut plus absorber toute cette énergie supplémentaire. Dans ce cas précis, la batterie, ne pouvant plus absorber davantage de courant, voit sa tension augmenter rapidement. Cette tension excessive est alors immédiatement transmise en retour vers le contrôleur. Le contrôleur doit alors encaisser cette surtension dans ses circuits internes. Ces composants ne sont généralement pas prévus pour supporter de telles tensions, ce qui peut provoquer leur destruction et conduire à un court-circuit interne.

Pour protéger efficacement le contrôleur de ce problème, il est nécessaire d'ajouter une résistance de freinage externe (brake resistor). Cette résistance permet d'absorber et de dissiper l'énergie excédentaire sous forme de chaleur, évitant ainsi les surtensions dangereuses.

Problème rencontré dans notre cas :

Notre contrôleur FSESC 4.12 ne dispose pas d'une sortie dédiée pour connecter directement une résistance de freinage (pas de connecteur spécifique de type "brake", "load", ou "regen resistor").

Solution adoptée : ODrive Regeneration Clamp

Pour résoudre ce problème, j'ai intégré un module externe appelé ODrive Regeneration Clamp dans le circuit de mon ergocycle embarqué. Ce composant est spécialement conçu pour détecter automatiquement les surtensions (par exemple, supérieures à environ 56 V dans le cas d'une batterie de 48 V nominale) et connecter automatiquement une résistance de freinage pour dissiper cet excès d'énergie électrique en chaleur.

Choix de la résistance de freinage

Mon système est alimenté par une batterie Li-ion 13S ayant une tension nominale de 48 V, et une tension maximale lorsqu'elle est entièrement chargée de 54,6 V.

Pour déterminer la puissance nécessaire que doit supporter cette résistance, j'ai utilisé la formule classique pour la dissipation thermique :

$$P = \frac{V^2}{R}$$

Avec une résistance de 5 Ω , cela donne :

$$P = \frac{54,6^2}{5} = \frac{2981,16}{5} = 596,2 \text{ W}$$

Ce résultat (environ 596 W) représente donc la puissance maximale qui serait dissipée par la résistance dans la pire situation possible : un freinage brutal alors que la batterie est totalement chargée, et donc incapable d'absorber davantage de courant.

La résistance que j'ai sélectionnée, une résistance en aluminium de 5 Ω – 300 W avec ailettes, possède une capacité spécifique de surcharge temporaire : elle peut supporter brièvement (2 à 5 secondes, durée typique d'un freinage) une puissance de 3 à 5 fois supérieure à sa puissance nominale. Cela signifie concrètement qu'elle peut tolérer des pics allant de 900 à 1500 W pendant quelques secondes.

Étant donné que mon calcul indique une puissance maximale d'environ 596 W, la résistance choisie offre une marge de sécurité importante, suffisante pour garantir une protection efficace du contrôleur contre toute surtension liée au freinage régénératif.



FIGURE 6 – Odrive Regen Clamp

3.1.6 Batterie Li-ion 48V – 10Ah

La batterie utilisée est une Li-ion 48V – 10Ah pour E-Bike.

Avec un moteur de 500W et une batterie Li-ion 48V – 10Ah (480 Wh), l'autonomie est estimée à :

$$\text{Autonomie} = \frac{480 \text{ Wh}}{500 \text{ W}} = 0,96 \text{ h} \approx \mathbf{58 \text{ minutes}}$$



3 Développement du matériel (hardware) de la version embarquée de l'ergocycle



FIGURE 7 – Batterie Li-ion 48V – 10Ah

3.1.7 Photos de la version embarquée finalisée de l'ergocycle



FIGURE 8 – Vue arrière de l'ergocycle

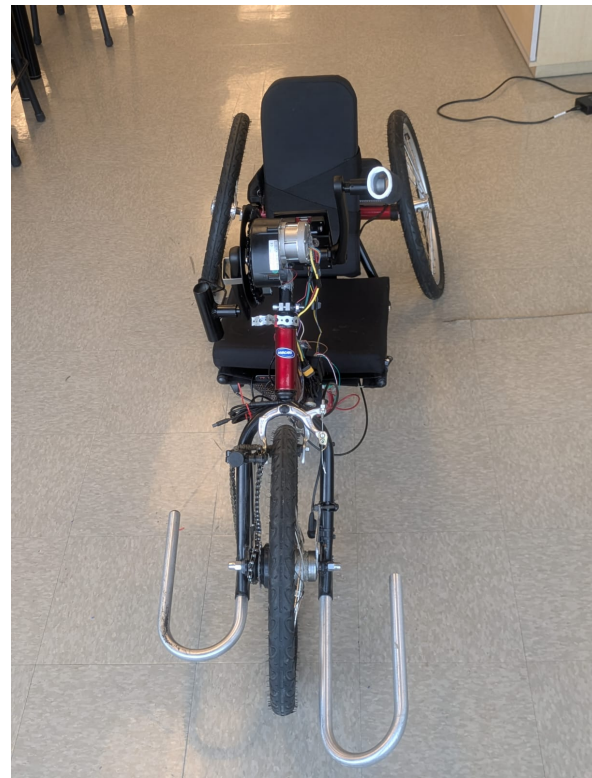


FIGURE 9 – Vue avant de l'ergocycle



3 Développement du matériel (hardware) de la version embarquée de l'ergocycle



FIGURE 10 – Disposition des composants électriques sur l'ergocycle



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

4.1 Choix initiaux

4.1.1 Passage de Python orienté objet à C++

Pour le développement de la version embarquée de l'ergocycle, j'ai choisi d'utiliser le langage de programmation C++. En effet, Python est un langage de haut niveau : il est simple à écrire et à comprendre, mais s'exécute lentement et utilise beaucoup de mémoire. Il est donc peu adapté à la programmation directe d'un microcontrôleur, où la vitesse et la consommation de ressources sont importantes. À l'inverse, C++ est un langage de plus bas niveau : il donne accès directement au matériel, permet d'écrire du code rapide et utilise peu de mémoire.

4.1.2 Justification du choix du microcontrôleur STM32F407G-DISC1

Le choix du microcontrôleur est un élément central, car il conditionne la capacité du système à traiter les données capteurs, gérer les protocoles de communication, et contrôler le moteur en temps réel.

J'ai d'abord choisi le microcontrôleur STM32F407VGT7TR pour les raisons suivantes :

- Il dispose de 192 Ko de RAM et 1 Mo de mémoire Flash, ce qui offre une grande marge pour structurer un projet embarqué.
- Interfaces de communication variées : Le STM32F407 dispose de plusieurs interfaces de communication UART, I2C et SPI qui permettent de communiquer avec les différents composants embarqués : le contrôleur de moteur, le lecteur de carte SD, et l'écran.
- De plus, ce microcontrôleur est pris en charge par STM32CubeIDE, ce qui facilite le développement, ainsi que la génération automatique de code pour les périphériques.

J'ai également commandé un ST-Link V2 afin de pouvoir compiler, flasher et déboguer le code sur la puce STM32.

Cependant, lors de la réception de la puce STM32F407VGT7TR seule, j'ai rencontré plusieurs difficultés techniques :

- La puce nécessitait une carte d'extension (breakout board) pour accéder aux broches de la puce. Ainsi des soudures très fines (au millimètre près) étaient nécessaires pour intégrer la puce sur le breakout board.
- Il fallait ajouter des convertisseurs de niveau logique pour assurer la compatibilité des tensions entre la puce et les différents périphériques. En effet, la puce STM32F407VGT7TR fonctionne en logique 3,3 V, or certains composants du projet ne sont pas directement compatibles : par exemple, l'écran Nextion utilisé ([voir 3](#)) nécessite une entrée en 5 V, tandis que le contrôleur de moteur VESC ([voir 3](#)) attend des signaux à 3,3 V strictement. Sans conversion de niveau, cela peut soit empêcher la communication, soit risquer d'endommager les composants. Cela nécessitait donc l'ajout de circuits intermédiaires pour adapter proprement les niveaux logiques, ce qui alourdissait encore le câblage et les contraintes d'intégration.



Pour pallier ces problèmes, j'ai décidé de basculer vers la carte de développement STM32F407G-DISC1, qui intègre déjà la puce STM32F407VGT6 (même cœur que la VGT7TR, avec un boîtier légèrement différent mais des performances identiques), ainsi que :

- un ST-Link intégré pour le débogage
- des broches facilement accessibles via headers,
- des convertisseurs de niveau intégrés
- une alimentation simplifiée via USB

Par la suite, au fil du développement, j'ai découvert le dépôt GitHub suivant :

https://github.com/vedderb/bldc_uart_comm_stm32f4_discovery

dans lequel Benjamin Vedder, le créateur du contrôleur de moteur VESC que j'utilise pour mon projet (voir 3), a développé un projet embarqué en utilisant le contrôleur de moteur VESC et le STM32F407G-DISC1. Cette découverte a conforté mon choix du microcontrôleur STM32F407G-DISC1.

4.1.3 Environnement de développement (STM32 Cube IDE)

Pour développer la version embarquée de l'ergocycle, j'utilise STM32CubeIDE, l'environnement officiel fourni par STMicroelectronics. Cet IDE regroupe à la fois un éditeur de code, un générateur de configuration STM32CubeMX (outil graphique de configuration), un compilateur et un débogueur intégré. Il est adapté au développement sur la carte STM32F407G-DISC1, que j'ai utilisée pour ce projet.

Création d'un projet en C++ sous STM32CubeIDE

Voici les étapes que j'ai suivies pour créer un nouveau projet en C++ dans STM32CubeIDE :

- **File → New STM32 Project** : Sélection de la carte de développement (STM32F407G-DISC1).

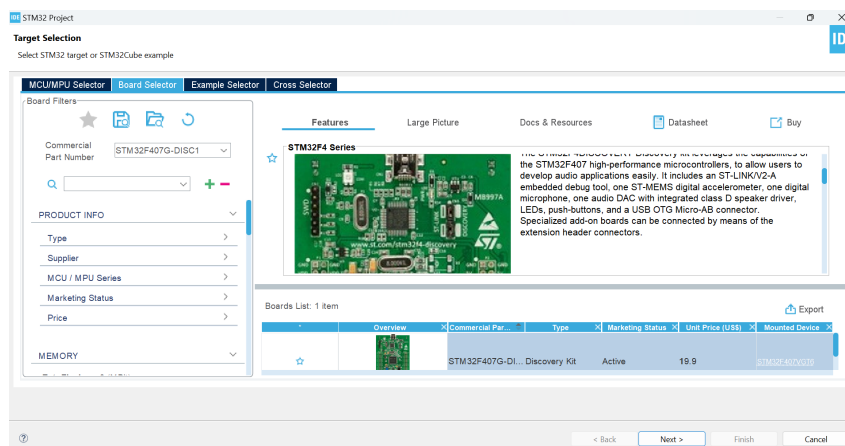


FIGURE 11 – Board Selector

- **Langage ciblé : C++**



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

Type de binaire : Executable : Je veux produire un code qui tourne sur la carte (et non une bibliothèque).

Type de projet : STM32Cube : Je souhaite bénéficier de la génération automatique de code pour les périphériques.

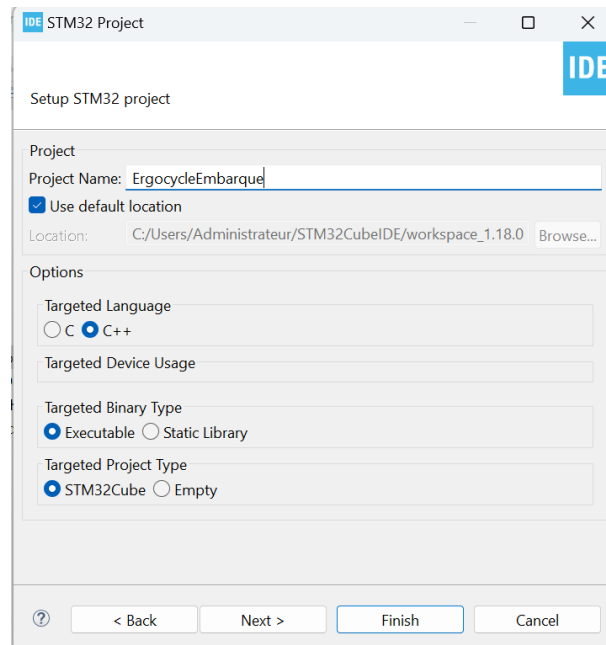


FIGURE 12 – Configuration du projet

- **Choix du mode d'initialisation des périphériques** : Lorsque l'on sélectionne une carte de développement dans STM32CubeIDE, une boîte de dialogue apparaît avec la question suivante : Initialize all peripherals with their default Mode ? J'ai cliqué sur "Yes", ce qui permet à STM32CubeMX de configurer automatiquement certains périphériques de la carte avec leurs valeurs par défaut. Cela permet de partir sur une base fonctionnelle que l'on peut ajuster par la suite.

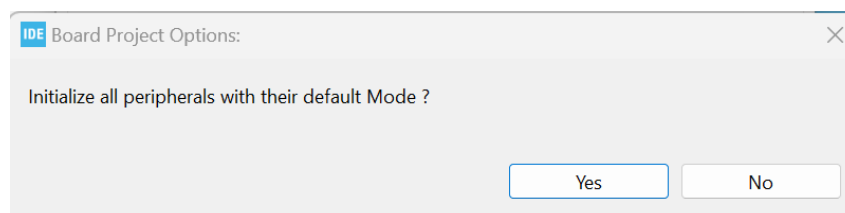


FIGURE 13 – Mode d'initialisation des périphériques par défaut

Remarque : Afin que le compilateur prenne correctement en charge le langage C++, il est nécessaire d'ajouter l'option suivante au compilateur :

`-std=c++17`

Cette option doit être ajoutée dans les paramètres du projet à l'emplacement suivant : Project Properties -> C/C++ Build -> Settings -> MCU/MPU GCC Compiler -> Miscellaneous



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

Dans la section **Miscellaneous**, il faut insérer `-std=c++17` dans le champ *Other flags*. Cela permet d'indiquer au compilateur d'utiliser le standard C++17, indispensable pour garantir la compatibilité avec certaines fonctionnalités modernes du langage utilisées dans le projet.

Configuration de la connexion entre le microcontrôleur et le contrôleur du moteur

Le contrôleur de moteur F5ESC (VESC) est connecté à l'interface USART3 du microcontrôleur STM32F407G-DISC1. Cette liaison série permet d'envoyer des consignes au moteur et de recevoir des informations en retour.

La vitesse de communication (baud rate) a été fixée à 115200 bauds dans STM32CubeMX et configurée à la même valeur dans VESC Tool, le logiciel utilisé pour paramétrer le firmware du contrôleur de moteur. Cette synchronisation des vitesses garantit une communication fiable entre le microcontrôleur et le VESC.

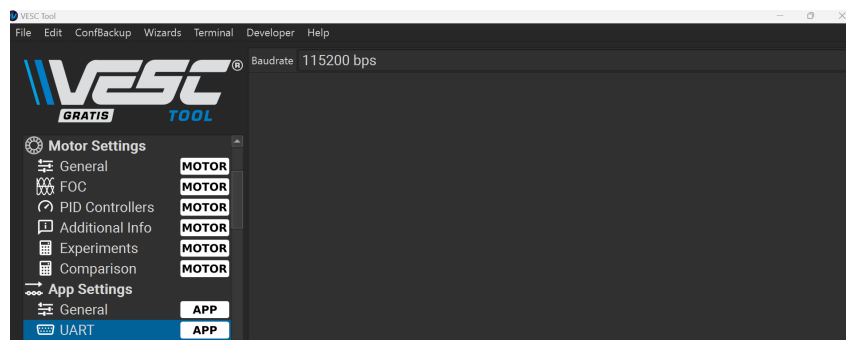


FIGURE 14 – Configuration de la vitesse de Baud dans VESC Tool

Configuration de la connexion entre le microcontrôleur et l'écran Nextion

L'écran est connecté à l'interface USART2 du microcontrôleur STM32F407G-DISC1. Cette liaison série permet d'envoyer des consignes au moteur et de recevoir des informations en retour.

La vitesse de communication (baud rate) a été fixée à 9600 bauds dans STM32CubeMX et configurée à la même valeur dans NextionEditor, le logiciel utilisé pour créer l'interface utilisateur qui sera affichée sur l'écran Nextion.

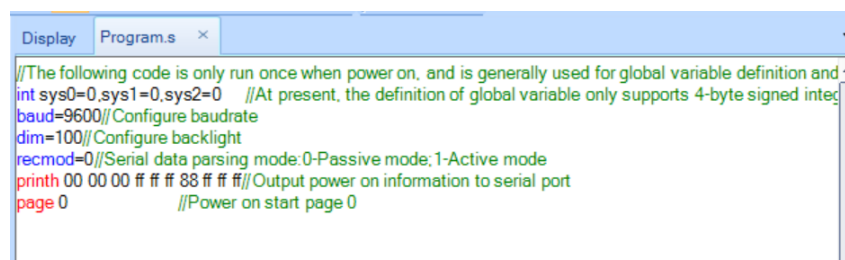


FIGURE 15 – Configuration de la vitesse de Baud dans Nextion Editor

Configuration du Watchdog (IWDG) dans STM32CubeIDE



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

Dans les systèmes embarqués, un watchdog est un mécanisme de sécurité matériel conçu pour détecter et réagir à des défaillances logicielles. Il s'agit d'un compteur autonome qui doit être régulièrement "rafraîchi" par le programme principal. Si le programme cesse de le faire – par exemple à cause d'un blocage ou d'une boucle infinie – le watchdog expire et provoque un redémarrage automatique du microcontrôleur. Cela permet de restaurer le système dans un état stable, sans intervention humaine.

Dans le cadre de ce projet, je vais utiliser le Independent Watchdog (IWDG). Ce watchdog est indépendant de l'horloge principale du système (il utilise son propre oscillateur interne LSI à 32 kHz), ce qui signifie qu'il continue de fonctionner même si le clock système subit une défaillance.

Le prescaler est configuré à 256 et la valeur de rechargement à 4095, ce qui (d'après la formule) permet de définir un délai maximal de surveillance avant redémarrage automatique du microcontrôleur. Ce délai, appelé timeout, est calculé selon la formule suivante :

$$\text{Timeout} = \frac{\text{Prescaler} \times \text{Reload Value}}{f_{\text{LSI}}} = \frac{256 \times 4095}{32000} \approx 32.76 \text{ secondes}$$

Ce délai représente le temps maximal avant redémarrage automatique du microcontrôleur.

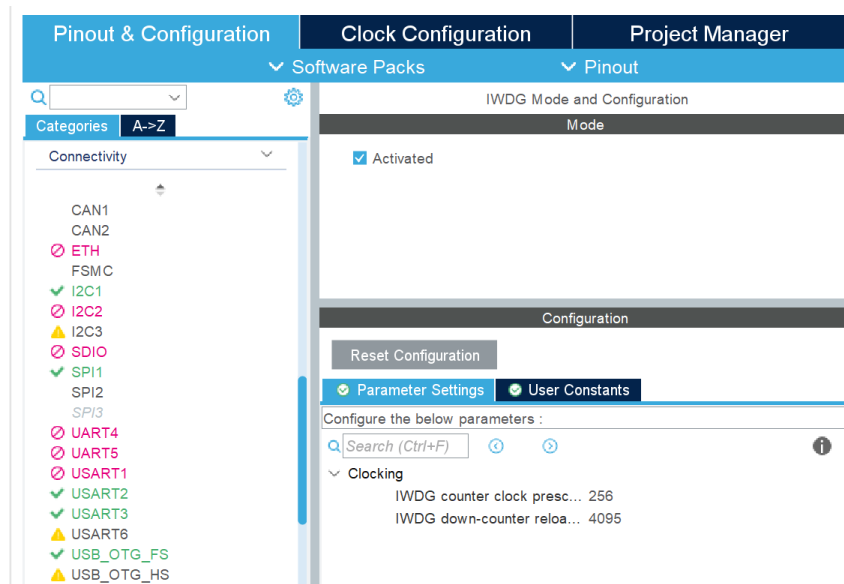


FIGURE 16 – Configuration du prescaler et de la formule de rechargement



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

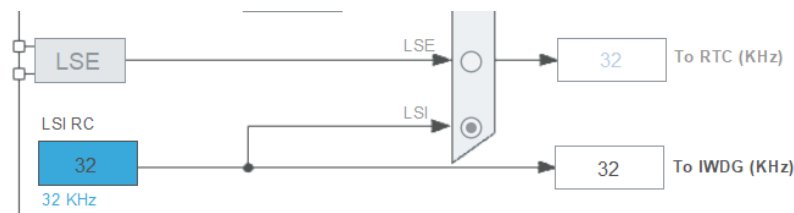


FIGURE 17 – Configuration de la fréquence de l'horloge LSI à 32KHz dans Clock Configuration

Une fois cette configuration effectuée, j'ai lancé la génération automatique du code via Projet -> Generate Code.

4.2 Architecture logicielle modulaire du projet

L'architecture logicielle est organisée en plusieurs modules indépendants : MotorController, VESCInterface, MotorComputations et ScreenDisplay.

Le module MotorController coordonne le système, il interroge le VESCInterface pour obtenir des informations en temps réel sur le moteur (cadence en RPM, courant, tension), fait appel à MotorComputations ([Voir MotorComputations](#)) pour calculer couple, puissance et vitesse angulaire à partir de la cadence en RPM, le courant, et la constante de couple. Dans l'autre sens, pour piloter le moteur, MotorController fait appel à VESCInterface et MotorComputations pour appliquer une cadence et un courant qui correspondent au couple, puissance ou vitesse attendu par l'utilisateur.

MotorController interagit également avec ScreenDisplay pour afficher les valeurs ou récupérer les consignes utilisateur (cadence, couple ou puissance désirés, mode de contrôle, etc.).

Le diagramme ci-dessous représente les interactions entre les différents modules logiciels (cadres rectangulaires) et les éléments matériels (cadres ovales) du système embarqué.

On peut suivre le cheminement des données en partant du module **VESC**, qui agit comme interface directe avec le moteur. Ce module transmet au microcontrôleur, via le module VESCInterface, les valeurs de courant, de cadence (RPM) et de tension : `getCurrent()`, `getRPM()`, `getVoltage()`.

Ces informations sont ensuite traitées par le module MotorController, qui les utilise directement ou les passe au module MotorComputations pour en déduire des grandeurs physiques, comme le couple ou la puissance (`getTorque()`, `getPower()`).

MotorController utilise ensuite la méthode `UpdateScreen()` pour envoyer les valeurs mesurées et calculées au module ScreenDisplay. Ce dernier les affiche sur l'écran grâce aux fonctions `showCadence()`, `showTorque()`, `showPower()`, etc.

L'utilisateur interagit également avec l'écran pour envoyer de nouvelles consignes (cadence, couple, puissance, direction, etc.). Ces données sont récupérées par le module ScreenDisplay

via la méthode `UpdateFromScreen()`, qui appelle des fonctions comme `getUserCadence()`, `getMode()` ou encore `getStop()`.

Ces consignes sont renvoyées au `MotorController`, qui les convertit si nécessaire en courant ou vitesse angulaire via `MotorComputations`, puis les applique au moteur en passant par le `VESCIInterface` avec les commandes `setCurrent()` ou `setRPM()`.

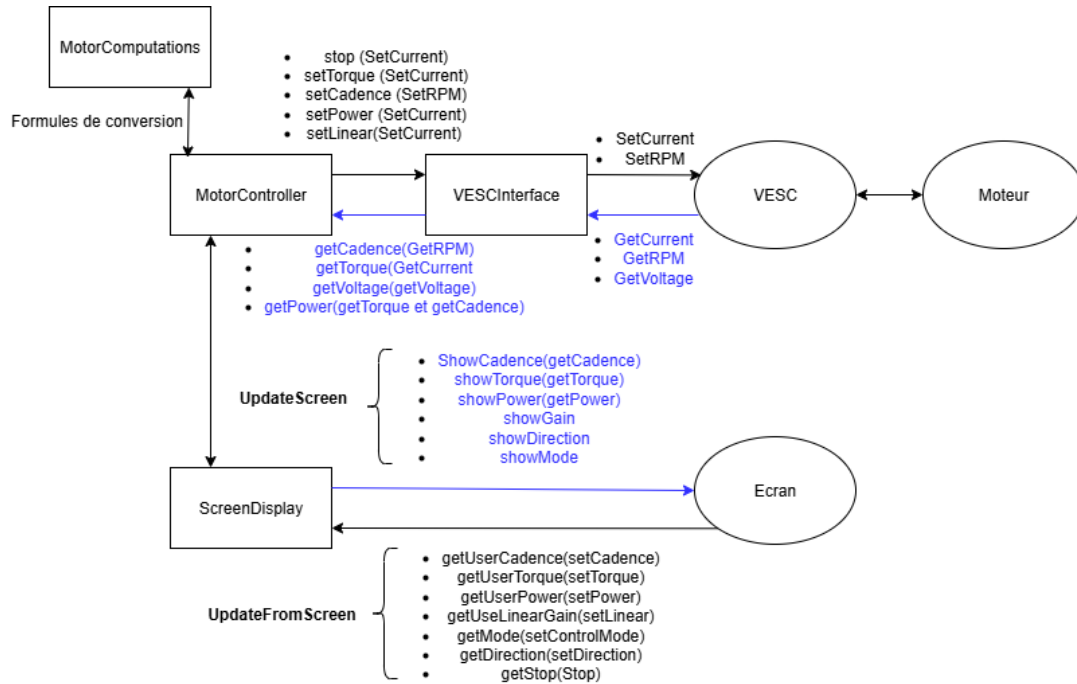


FIGURE 18 – Architecture modulaire du code du projet

[Voir dépôt GitHub du code de l'ergocycle embarqué](#)

4.3 Méthode de calibration de la constante de couple K_t du moteur dans le projet

La constante de couple K_t relie le courant traversant un moteur à l'effort de couple qu'il produit : τ (en $N \cdot m$) = $K_t \cdot I$ (en A). K_t s'exprime en $N \cdot m/A$.

La fonction `MotorController::calibrateTorqueConstant()` est responsable de l'estimation de la constante de couple K_t du moteur, exprimée en $N \cdot m/A$. L'idée générale est la suivante : on applique un courant connu au moteur (5 A), on laisse le moteur réagir pendant un court instant, puis on mesure le couple résultant. En divisant le couple mesuré par le courant appliqué, on obtient une estimation de K_t via la formule :

$$K_t = \frac{\tau}{I}$$



4.4 Gestion de la batterie

Afin de fournir à l'utilisateur une indication claire sur l'état de charge de la batterie de l'ergocycle, une fonctionnalité de lecture de la tension a été implémentée dans le module VESCInterface, via la méthode `getVoltage()`, qui interroge le VESC pour récupérer la tension en sortie de la batterie. Cette valeur est ensuite transmise au module MotorController, qui l'analyse afin de calculer un pourcentage de charge estimé, et d'afficher à l'écran un message d'alerte ou d'information en fonction du niveau détecté.

Le système utilise une batterie Li-ion 13S, dont la tension maximale en pleine charge est de 54,6 V. Le contrôleur de moteur a été configuré dans VESC Tool (voir Figure 9) pour :

- réduire automatiquement sa puissance en dessous de 44,2 V
- s'arrêter complètement si la tension descend sous 40,3 V

ce qui permet de protéger la batterie contre une décharge excessive et d'augmenter sa durée de vie.

The screenshot displays the VESC Tool configuration window. The top section contains five rows of settings, each with a label, a value, and control icons (up/down arrows, a save icon, and a help icon). The values are: Battery Voltage Cutoff Start (44,2 V), Battery Voltage Cutoff End (40,3 V), Battery Voltage Regen Cutoff Start (1000,0 V), Battery Voltage Regen Cutoff End (1100,0 V), and Battery Filter Constant (45). Below this is a 'Battery Cutoff Calculator' section with a dropdown menu set to 'Lithium Ion (full at 4.2V/cell)', a 'Cells' input set to 13, and a resulting range '[44.20, 40.30]'. An 'Apply' button is at the bottom right of the calculator section.

Parameter	Value
Battery Voltage Cutoff Start	44,2 V
Battery Voltage Cutoff End	40,3 V
Battery Voltage Regen Cutoff Start	1000,0 V
Battery Voltage Regen Cutoff End	1100,0 V
Battery Filter Constant	45

Battery Cutoff Calculator

Type: Lithium Ion (full at 4.2V/cell)

Cells: 13

[44.20, 40.30]

Apply

FIGURE 19 – Configuration des seuils de fonctionnement du contrôleur sur VESC Tool

4.5 Installation du firmware du contrôleur VESC

Le contrôleur utilisé dans ce projet est un Flipsky F5ESC 4.12 50A, sa configuration nécessite l'installation du logiciel Flipsky VESC Tool : https://vesc-project.com/vesc_tool

Une fois VESC Tool installé et le contrôleur connecté en USB à l'ordinateur, la fonction Autoconnect (accessible via l'onglet Connection dans le menu latéral gauche) permet à l'outil de détecter automatiquement le VESC, en vue de sa configuration.

La configuration du firmware débute ensuite par l'assistant Setup Motors FOC, accessible depuis l'onglet Welcome Wizards. Cet assistant guide l'utilisateur pas à pas dans le paramétrage du contrôleur.



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

Le moteur utilisé dans ce projet étant un BLDC TSDZ2 48V/500W (800 g), les figures suivantes présentent, étape par étape, les paramètres sélectionnés afin d'optimiser la compatibilité entre le contrôleur et le moteur.

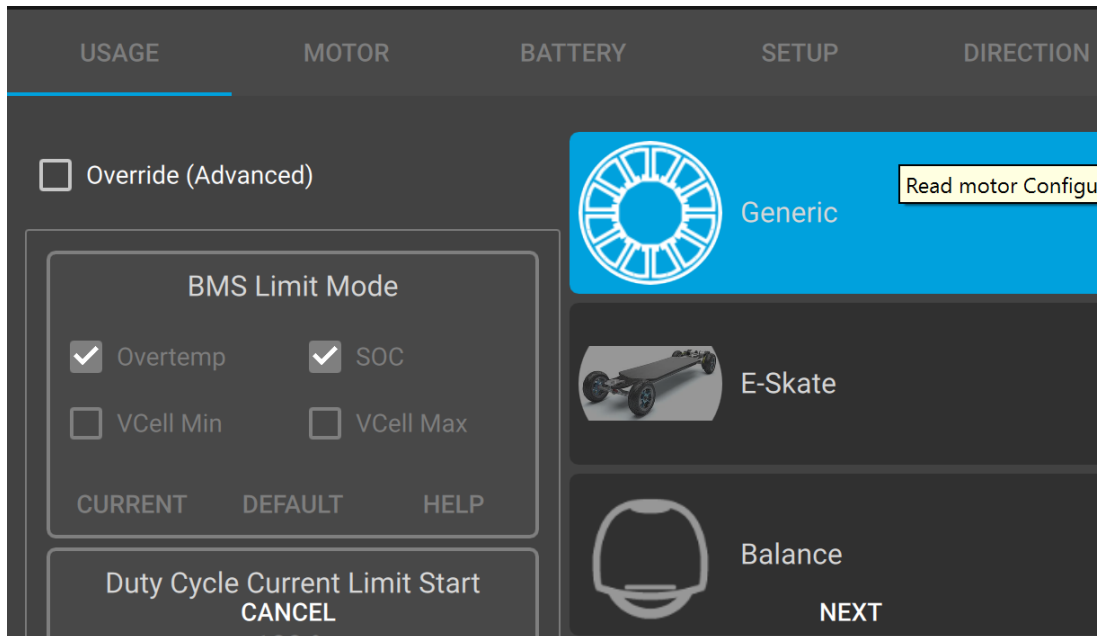


FIGURE 20 – Etape 1 : usage

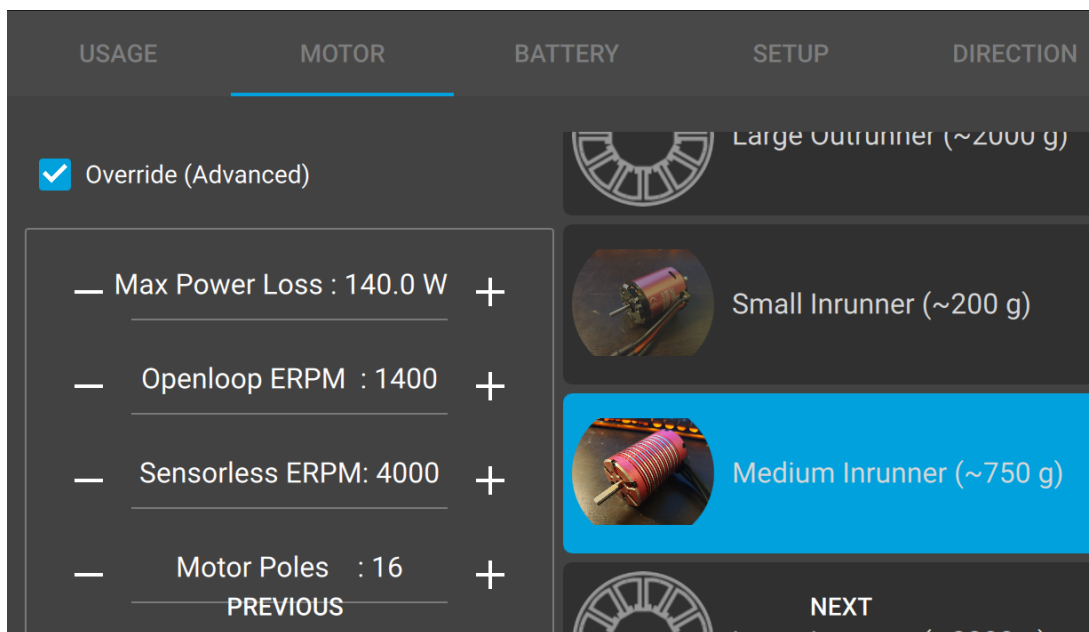


FIGURE 21 – Etape 2 : Motor



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

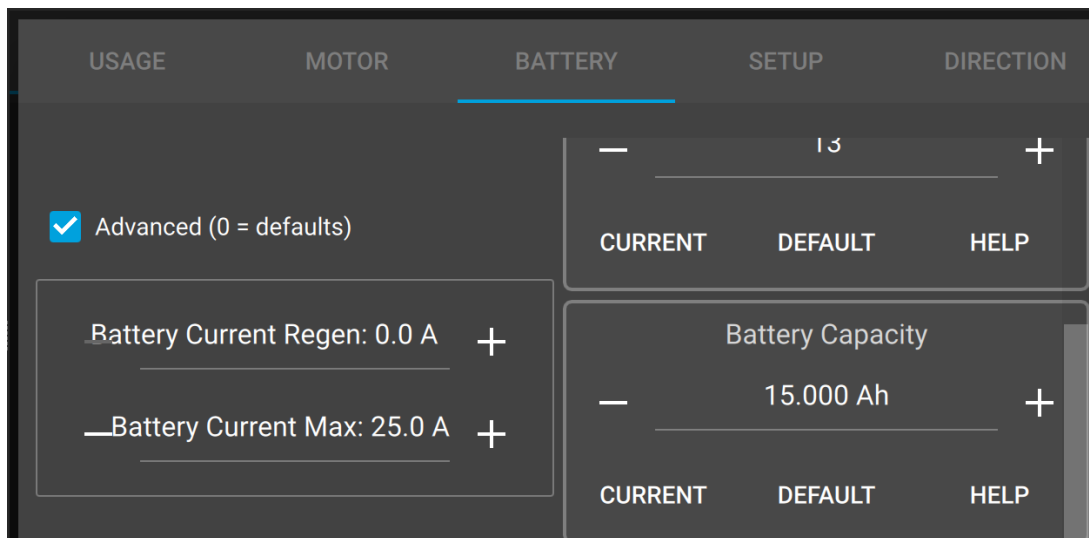


FIGURE 22 – Etape 3 : Battery

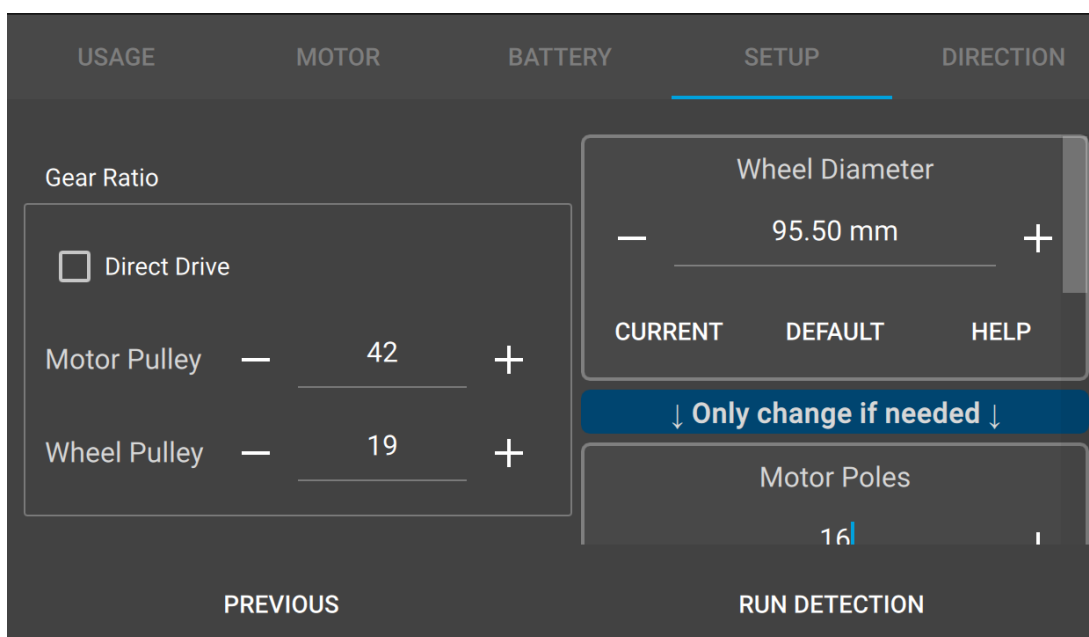


FIGURE 23 – Etape 4 : Setup

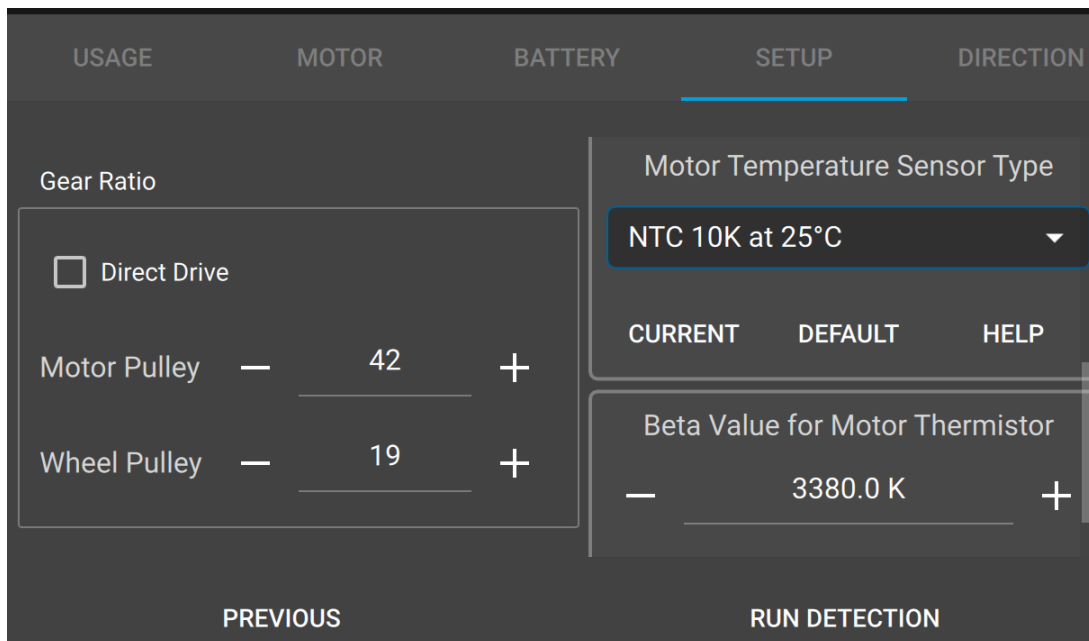


FIGURE 24 – Etape 4 : Setup

4.6 Conception de l'interface utilisateur graphique (GUI) de l'écran

L'interface est conçue sur le logiciel : [NextionEditor](#).

NextionEditor permet de concevoir une interface utilisateur en plaçant visuellement **des objets** (boutons, champs texte, etc.)

Pour illustrer concrètement le fonctionnement des objets dans Nextion Editor et leur interaction avec le code embarqué, prenons l'exemple de l'affichage de la cadence du moteur sur l'écran.

Dans Nextion Editor, on ajoute sur une page un objet de type text destiné à afficher la cadence en temps réel. Cet objet est nommé par exemple `cad_val` dans le champ `objname` de la barre latérale à droite. Ce nom sera utilisé pour référencer l'objet dans le code.



4 Développement du logiciel (software) de la version embarquée de l'ergocycle

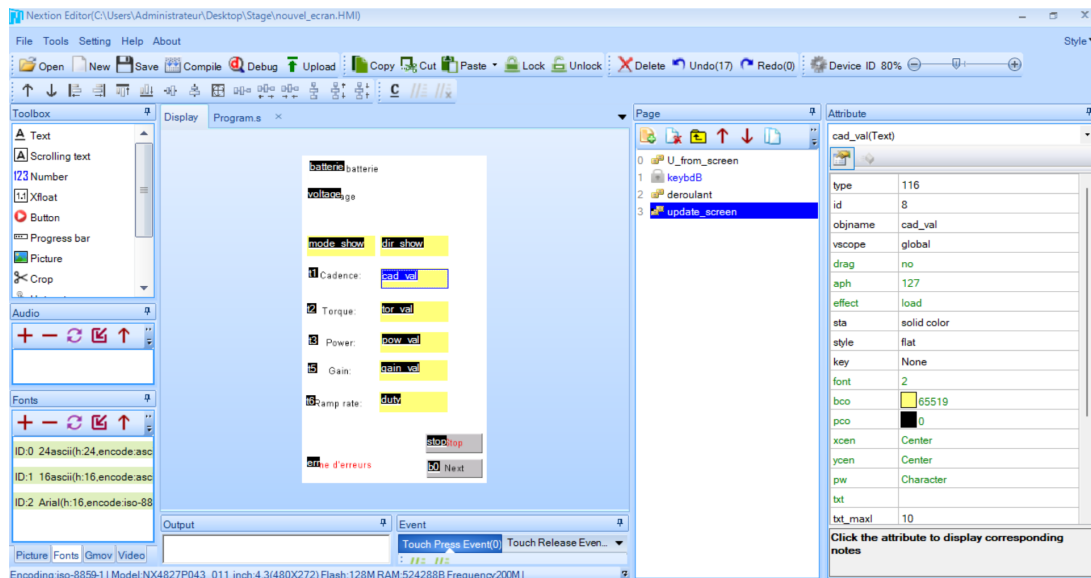


FIGURE 25 – Interface de NetionEditor

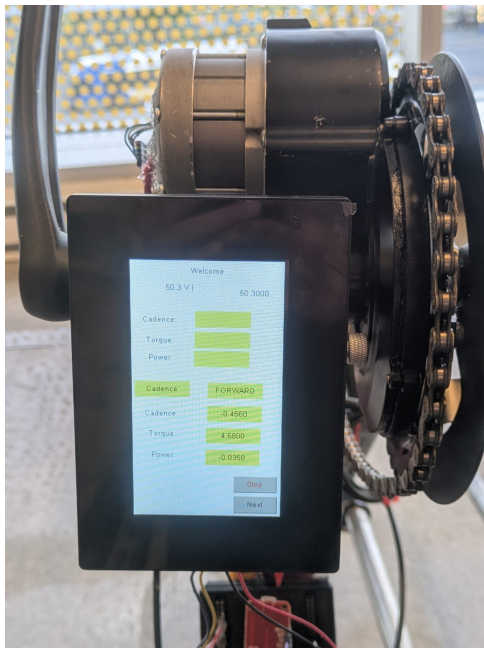
L'interface que j'ai conçue comporte trois pages :

- **Page 1** : sélection avec un clavier numérique de la consigne — cadence, couple, puissance ou gain, selon le mode choisi et affichage du mode et du sens sélectionnés ainsi que des données — cadence, couple, puissance et gain en temps réel.
- **Page 2** : choix du *mode* à l'aide d'un menu déroulant.
- **Page 3** : sélection du *sens de rotation* (avant/arrière) via un menu déroulant.

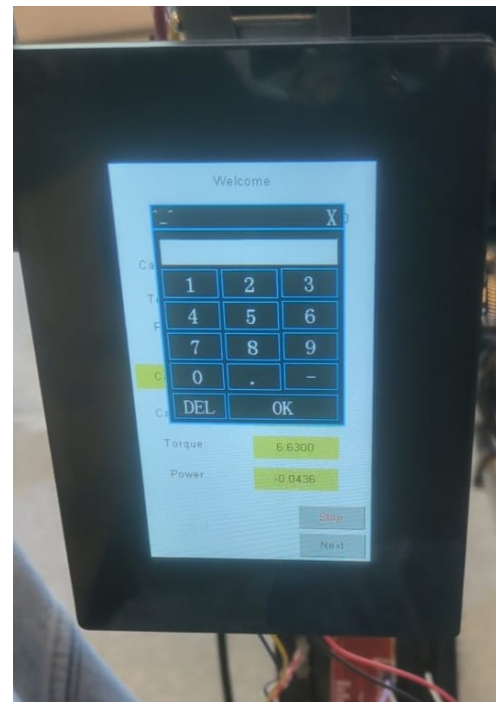
Toutes les pages affichent également le niveau de batterie et la première page comporte un bouton STOP pour arrêter le moteur.



4 Développement du logiciel (software) de la version embarquée de l'ergocycle



Page 1

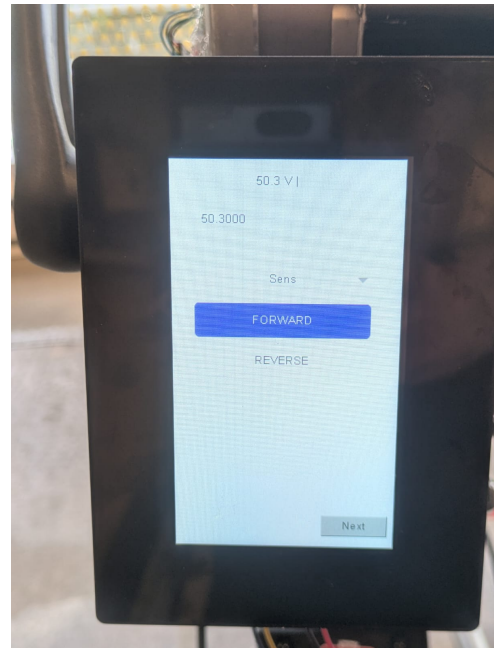


Clavier numérique permettant la saisie de la consigne

FIGURE 26 – Interface utilisateur : pages 1



Page 2



Page 3

FIGURE 27 – Interface utilisateur : pages 3 et 4

Côté microcontrôleur, le module ScreenDisplay encapsule la communication UART avec l'écran. Pour envoyer la cadence à l'objet `cad_val`, on utilise une fonction dédiée :

ScreenDisplay ::showCadence

```
void ScreenDisplay::showCadence(float rpm) {
    sendValue("cad_val", rpm, "%.1f");
}
```

Cette méthode (`showCadence`) appelle fonction de la classe `ScreenDisplay` nommée `sendValue()`. Cette dernière prépare la commande `cad_val` au format texte que l'écran `Nextion` peut interpréter via son protocole série.

La commande finale transmise via UART

```
cad_val.txt="74.5" 0xFF 0xFF 0xFF
```

Les trois octets spéciaux marquent la fin de la commande. Ce sont trois fois la valeur `0xFF` (appelés "end-of-command bytes" dans la documentation officielle de `Nextion`).

Ainsi, lorsque le programme calcule la cadence réelle du moteur (par exemple dans `MotorController` via `getRPM()`), il peut simplement appeler :

ScreenDisplay ::showCadence

```
screen->showCadence(rpm);
```

Et la valeur sera immédiatement affichée dans le champ texte `cad_val` de l'interface graphique.

Le fonctionnement des objets est le même quand il s'agit de l'envoi d'informations de l'écran vers le microcontrôleur.

L'ensemble du projet est compilé dans un fichier `.tft`, qui est ensuite téléversé sur l'écran Nexion via carte SD ou port série. J'ai utilisé une carte [Micro SD 4 GB](#). Attention à utiliser une carte FAT32 uniquement et à bien penser à débrancher l'écran du microcontrôleur pour insérer et retirer la carte.

Remarque : tous les objets de l'interface doivent être déclarés avec `vscop` global sur la barre latérale à droite de NexionEditor pour fonctionner correctement.

5 Tests du système

5.1 Problème de liaison entre le pédalier le moteur

Une fois les aspects logiciels et matériels décrits ci-dessus finalisés, j'ai procédé à une série de tests afin de vérifier que le système fonctionne conformément aux attentes.

Une première remarque concerne le comportement du moteur lorsqu'il est utilisé en mode cadence. Contrairement à l'ergocycle non embarqué (projet existant), j'ai remarqué que dans cette version embarquée, les pédales ne sont pas entraînées par le moteur. Cela signifie que lorsque le moteur est activé, il ne transmet pas de mouvement aux pédales, et inversement, les pédales n'exercent aucun effet sur le moteur. Cette déconnexion mécanique a plusieurs conséquences.

Tout d'abord, en mode cadence, le contrôleur se base uniquement sur la vitesse de rotation du moteur, sans prendre en compte la cadence réellement appliquée par l'utilisateur. Cela fausse l'objectif de ce mode, qui est justement de synchroniser ou d'assister l'utilisateur en fonction de son propre rythme.

Ensuite, en mode couple (torque), le système ne peut pas détecter le couple que l'utilisateur applique sur les pédales, ce qui empêche le contrôleur de réagir en conséquence.

Enfin, cela impacte directement les modes `power concentric` et `power eccentric`, qui nécessitent de connaître la cadence de l'utilisateur pour calculer le couple moteur à appliquer en fonction d'une puissance cible.

Des pièces usinées ont donc été ajoutées au projet afin de forcer la transmission du mouvement du moteur vers le pédalier. Ces modifications ont déjà été effectuées par les chercheurs qui

ont travailler sur le développement de la version non embarquée de l'ergocycle.

5.2 Latences

Une fois le problème d'entraînement du pédalier par le moteur résolu, j'ai procédé à un second test fonctionnel de l'ergocycle embarqué, toujours dans le mode «cadence», car il est le plus simple à visualiser concrètement lors des essais. À cette étape, j'ai observé une latence importante entre le moment où je sélectionne une cadence précise sur l'écran tactile et le moment où cette consigne est réellement prise en compte par le moteur. J'ai également constaté ce même retard lors de l'appui sur la commande «stop», ainsi que lors de la mise à jour des valeurs transmises du système vers l'écran (niveau de batterie, cadence, couple appliqué par l'utilisateur, puissance appliquée par l'utilisateur, direction, mode ...). Ces délais peuvent nuire à la fluidité d'utilisation de l'interface.

Jusqu'à présent, l'ergocycle était commandé par le module `main.cpp`, qui mettait à jour l'écran avec les informations du système via la fonction `updateScreen()` (définie dans `MotorController.cpp`), puis récupérait les consignes de l'utilisateur depuis l'écran et les exécutait via la fonction `updateFromScreen()`. Ces deux fonctions s'alternaient dans une boucle infinie (`while(1)`), permettant à l'utilisateur de contrôler l'ergocycle et de visualiser en temps réel les variables telles que la batterie, la cadence, le couple ou encore la puissance appliqués. Toutefois, cette approche n'est pas suffisamment rapide pour assurer un pilotage instantané et fluide de l'ergocycle.

5.2.1 Installation de l'OS FreeRTOS

J'ai donc résolu ce problème en installant l'OS FreeRTOS sur le microcontrôleur. FreeRTOS permet d'exécuter simultanément plusieurs tâches indépendantes, réduisant ainsi significativement les latences observées. J'ai créé deux tâches distinctes : une première tâche **TaskUpdateFromS** se charge de la mise à jour du système à partir des consignes entrées par l'utilisateur, tandis qu'une seconde tâche **TaskUpdateS** gère exclusivement l'actualisation des données affichées sur l'écran. En effet, la tâche `TaskUpdateFromS` est associée à la fonction `StartTaskUpdateFromS`, qui exécute la méthode `updateFromScreen()`, chargée de lire les consignes utilisateur. De son côté, la tâche `TaskUpdateS` est liée à la fonction `StartTaskUpdateS`, qui appelle `updateScreen()` afin de mettre à jour l'affichage des informations système sur l'écran. Cette architecture multitâche garantit un pilotage fluide et réactif de l'ergocycle.

La tâche `TaskUpdateFromS` a une priorité plus élevée que `TaskUpdateS`.

Pour le mode linéaire, j'ai ajouté une troisième tâche dédiée **LinearControl**, qui vérifie en continu si ce mode est sélectionné. Lorsque le mode linéaire est actif, cette tâche lit en temps réel la cadence mesurée et applique la loi de commande linéaire en appelant périodiquement la méthode `update()`. Ainsi, le couple moteur est ajusté dynamiquement en fonction de la cadence réelle, comme expliqué dans 2.4.

C'est désormais le module `FreeRTOS.cpp` qui contient la définition de ces tâches et leur exécution.



6 Exploitation des signaux électromyographiques (EMG) pour adapter l'assistance de l'ergocycle embarqué et prolonger l'effort musculaire de l'utilisateur

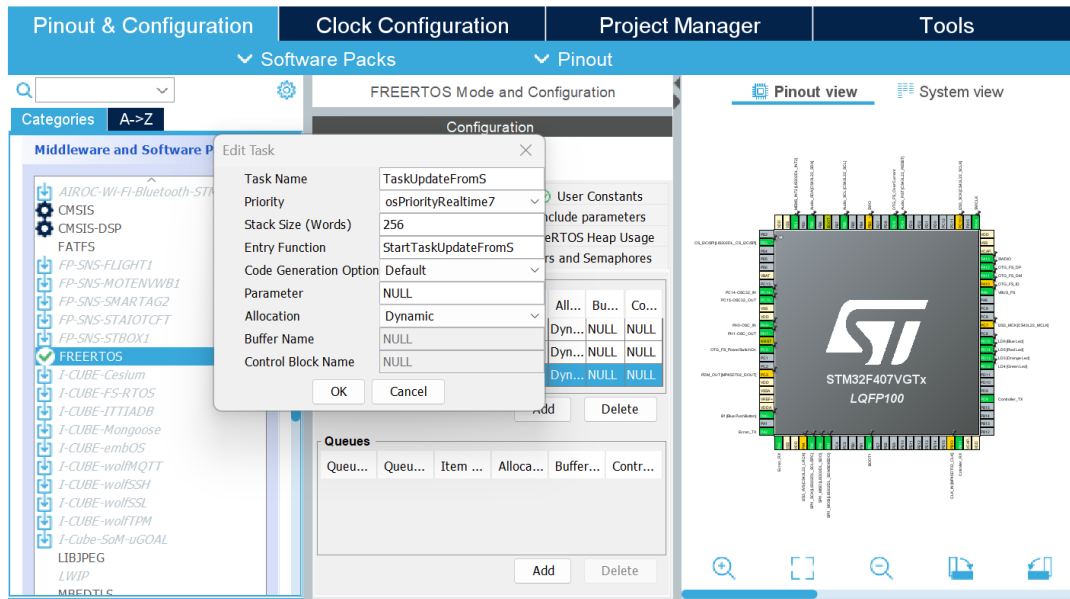


FIGURE 28 – Création de la tâche TaskUpdateFromS

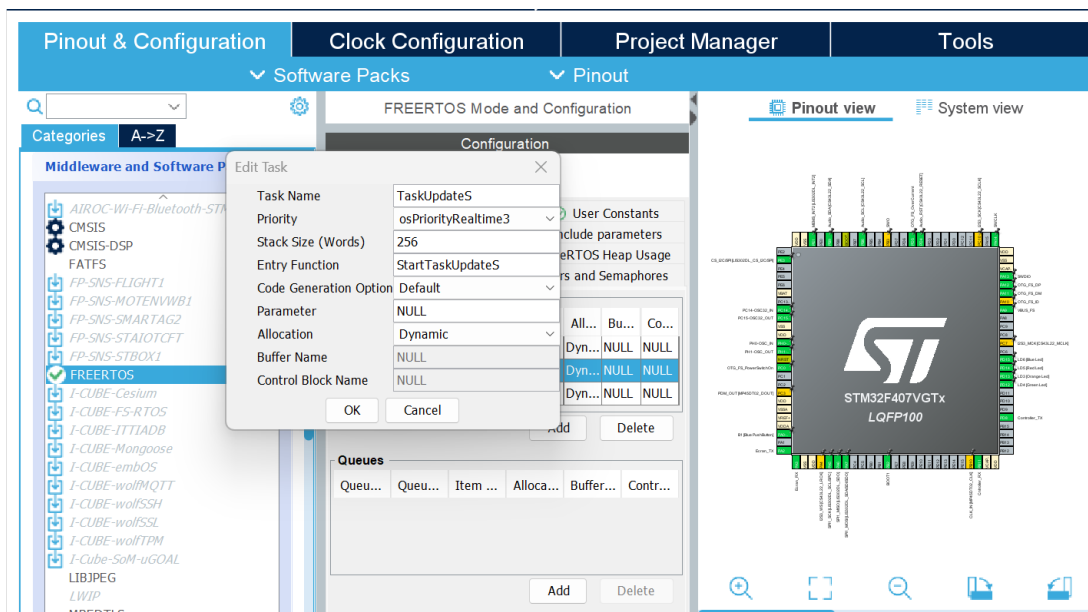


FIGURE 29 – Création de la tâche TaskUpdateS

6 Exploitation des signaux électromyographiques (EMG) pour adapter l'assistance de l'ergocycle embarqué et prolonger l'effort musculaire de l'utilisateur

Maintenant que l'ergocycle fonctionne comme prévu, j'aimerais en faire évoluer le comportement. Mon objectif est de suivre l'évolution des signaux EMG provenant des biceps de l'utilisateur tout au long de la séance, afin d'adapter dynamiquement le mode de fonctionnement



initialement sélectionné. Ceci permettra de maximiser le temps de l'exercice et de retarder le plus possible l'épuisement musculaire.

J'ai ainsi développé un modèle de machine learning capable de prédire en temps réel, à partir des signaux EMG du biceps, le temps restant avant l'épuisement musculaire. J'ai ensuite intégré ce modèle dans le système embarqué afin que l'ergocycle puisse ajuster automatiquement la consigne du mode en cours lorsque l'épuisement est imminent.

Concrètement, dès que le modèle prédit qu'il reste environ une minute avant l'épuisement, une adaptation automatique de la consigne est déclenchée, selon le mode actif :

- Mode Cadence : la cadence cible est réduite de 20%
- Mode Torque :
 1. en sens avant (forward), le couple est augmenté de 20%
 2. en sens arrière (reverse), il est réduit de 20%
- Mode Power Concentric : la puissance est augmentée de 20%
- Mode Power Eccentric : la puissance est réduite de 20%
- Mode Linéaire :
 1. en sens avant (forward), le gain est augmenté de 20%
 2. en sens arrière (reverse), le gain est réduit de 20%

[Voir le code de l'ergocycle embarqué qui intègre les signaux EMG dans son pilotage.](#)

6.1 Première étape : développement d'un modèle de prédiction du temps restant avant l'épuisement musculaire.

[Lien vers le jupyter notebook du modèle.](#)

Mon but étant d'entraîner un modèle capable de prédire en temps réel la durée restante avant l'épuisement musculaire, à partir des signaux EMG des biceps de l'utilisateur, de son niveau de fatigue musculaire et du temps écoulé depuis le début de l'exercice, il me fallait une base de données constituée de séries temporelles EMG enregistrées au niveau du biceps, annotées en fonction du niveau de fatigue correspondant, afin d'entraîner mon modèle dans des conditions réalistes.

Pour cela, j'ai utilisé le jeu de données proposé dans l'article "[A Comprehensive Dataset of Surface Electromyography and Self-Perceived Fatigue Levels for Muscle Fatigue Analysis](#)" (Cerqueira et al., 2024). Ce dataset regroupe plus de 13 heures de signaux sEMG enregistrés auprès de 13 participants, lors de l'exécution de douze mouvements dynamiques des membres supérieurs, répartis entre des mouvements mono-articulaires (comme la flexion du coude, l'extension du coude, l'abduction et la flexion de l'épaule) et des mouvements composés simulant des tâches industrielles (notamment un mouvement de type soudure).



6.1.1 Architecture de la base de données

Chaque participant dispose d'un dossier contenant ses propres essais, chaque essai correspondant à un mouvement. Pour chaque essai, deux fichiers sont fournis : un fichier .csv contenant les signaux sEMG bruts, enregistrés à une fréquence de 1259 Hz, et un autre fichier .csv contenant l'évolution du niveau de fatigue perçu au cours de l'exercice sur une échelle de 0 à 2, échantillonné à 50 Hz. J'ai décidé de me concentrer sur les données EMG enregistrées au niveau du biceps, en sélectionnant les mouvements où ce muscle est principalement sollicité, afin de garantir la cohérence entre les données d'entrée du modèle et l'objectif de prédiction ciblée sur l'épuisement du biceps.

6.1.2 Prtraitement des données

1. Filtrage et amplification du signal

J'ai d'abord amplifié chaque signal EMG brut (gain de 200) puis filtré à l'aide d'un filtre passe-bande Butterworth du premier ordre, avec une fréquence de coupure basse à 20,8 Hz et une fréquence de coupure haute à 498,4 Hz. Ce filtrage permet de ne conserver que les composantes fréquentielles pertinentes du signal musculaire, en éliminant le bruit basse et haute fréquence. J'ai choisi ces valeurs de fréquences de coupure et ce gain car le capteur EMG que j'utiliserai lors de l'intégration sur l'ergocycle applique exactement le même filtrage et la même amplification. Cela me permet d'assurer une cohérence entre les conditions d'entraînement du modèle et celles de son application réelle.

2. Découpage en fenêtres glissantes

J'ai ensuite découpé les signaux filtrés en fenêtres temporelles de 4 secondes, avec un recouvrement de 2 secondes entre chaque fenêtre. Ce découpage permet d'extraire des segments de signal sur lesquels seront calculées les caractéristiques (features). J'ai choisi une durée de 4 secondes car cela correspond à la durée d'un mouvement complet du bras (2 secondes de flexion, 2 secondes d'extension), comme précisé dans l'article de référence.

3. Extraction de caractéristiques (features)

Pour chaque fenêtre, plusieurs caractéristiques temporelles et fréquentielles sont extraites afin de décrire la structure du signal EMG :

— RMS (Root Mean Square)

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

C'est une mesure de la puissance moyenne du signal. Elle est sensible à l'amplitude des signaux EMG. Lorsque le muscle fatigue, plus d'unités motrices sont recrutées pour maintenir l'effort. Cela provoque une augmentation de l'amplitude du signal, donc du RMS.



— **MAV (Mean Absolute Value)**

$$\text{MAV} = \frac{1}{N} \sum_{i=1}^N |x_i|$$

La MAV correspond à la moyenne des valeurs absolues du signal. Elle reflète l'amplitude globale du signal EMG, et a tendance à augmenter en cas de fatigue, de façon similaire au RMS.

— **ZC (Zero Crossing)**

Le nombre de passages par zéro est donné par :

$$\text{ZC} = \sum_{i=1}^{N-1} \mathbb{I}[(x_i \cdot x_{i+1}) < 0]$$

où $\mathbb{I}$ est la fonction indicatrice. Cette mesure correspond au nombre de fois où le signal change de signe. Quand un muscle se fatigue, la vitesse de conduction des fibres musculaires diminue. Cela provoque des oscillations plus lentes du signal EMG. Donc, moins de passages par zéro.

— **WL (Waveform Length)**

$$\text{WL} = \sum_{i=1}^{N-1} |x_{i+1} - x_i|$$

La WL mesure la longueur totale de la courbe, représentant l'activité globale et la complexité du signal. Elle a tendance à augmenter avec la fatigue musculaire.

— **MDF (Median Frequency)**

La fréquence médiane f_{mdf} est la fréquence qui divise le spectre de puissance en deux parties égales :

$$\sum_{f=f_{\min}}^{f_{\text{mdf}}} P(f) = \frac{1}{2} \sum_{f=f_{\min}}^{f_{\max}} P(f)$$

où $P(f)$ est la densité spectrale de puissance du signal. Avec la fatigue, le MDF diminue généralement, reflétant un glissement du contenu fréquentiel vers les basses fréquences.

— **MNF (Mean Frequency)**

$$\text{MNF} = \frac{\sum_{i=1}^N f_i \cdot P(f_i)}{\sum_{i=1}^N P(f_i)}$$

La MNF est la moyenne pondérée des fréquences par leur densité spectrale de puissance. Elle diminue aussi lors de la fatigue musculaire.

5. Étiquetage par la fatigue perçue

J'ai ensuite associé les fenêtres à un label de fatigue, issu d'un fichier séparé contenant l'évolution de la fatigue perçue au cours du temps. Pour chaque fenêtre, le label de fatigue est déterminé en fonction du temps central de la fenêtre, en prenant la valeur la plus proche dans le fichier de fatigue.

6. Calcul de la durée restante et écoulée

J'ai ajouté deux nouvelles colonnes pour chaque fenêtre :



- `duration_remaining` : durée restante avant la fin de l'essai (calculée à partir du temps central de la fenêtre).
- `duration_since_start` : durée écoulée depuis le début de l'essai (temps central de la fenêtre).

6.1.3 Développement et entraînement du modèle

J'ai choisi d'utiliser un réseau de neurones récurrent de type **LSTM** (Long Short-Term Memory). Un LSTM est un modèle adapté à l'analyse de séquences temporelles : il est capable de mémoriser des informations sur plusieurs instants successifs, ce qui le rend idéal pour traiter des signaux physiologiques comme l'EMG, où la dynamique dans le temps est essentielle. Dans mon cas, chaque entrée du modèle correspond à une séquence de 5 fenêtres temporelles consécutives, chacune décrite par les features que j'ai présenté dans 6.1.2 (RMS, MAV, ZC, WL, MDF, MNF, `fatigue_label`, `duration_since_start`). La variable cible que je cherche à prédire est la `duration_remaining`, c'est-à-dire la durée restante avant la fin de l'effort.

Pour construire mon modèle, j'ai utilisé l'API Keras de TensorFlow. J'ai défini une architecture simple : une couche LSTM de 32 unités, suivie d'une couche de dropout (taux 0,3) pour limiter le surapprentissage, puis d'une couche Dense de sortie adaptée à la régression. J'ai compilé le modèle avec l'optimiseur Adam (learning rate 0,001) et la fonction de perte MSE (Mean Squared Error). J'ai ensuite entraîné le modèle sur mes données d'entraînement (80 % de la base de données), en utilisant un mécanisme d'early stopping pour éviter le surapprentissage : si la performance sur l'ensemble de validation ne s'améliore pas pendant 20 époques, l'entraînement s'arrête automatiquement et les meilleurs poids sont restaurés. J'ai utilisé un batch size de 64 et un maximum de 1500 époques.

Après l'entraînement, j'ai évalué les performances du modèle à l'aide de plusieurs métriques : MAE (erreur absolue moyenne), RMSE (racine de l'erreur quadratique moyenne) et R^2 (coefficient de détermination), aussi bien sur l'ensemble d'entraînement que sur l'ensemble de test. Enfin, j'ai visualisé la qualité des prédictions à l'aide d'un graphique de dispersion : plus les points sont proches de la diagonale, plus la prédiction de la durée restante est précise.

6.1.4 Résultats

Sur les données de test, j'obtiens un MAE de 18,62s, un RMSE de 30,47s et un R^2 de 0,90. Sur les données d'entraînement, le MAE est de 14,41s, le RMSE de 22,18s et le R^2 de 0,95.

Pour rappel, le R^2 varie normalement entre -1 et 1, une valeur de 1 correspondant à un modèle parfait. Mes résultats, avec des R^2 proches de 1, montrent donc que mon modèle prédit très bien la durée restante.

De plus, l'écart relativement faible entre les performances sur l'entraînement et le test indique que je n'ai pas de problème d'overfitting : mon modèle généralise bien sur de nouvelles données.

Les graphiques de dispersion des ensembles de validation et de test (ci-dessous) montrent que les points sont globalement alignés sur la diagonale, ce qui confirme la précision du modèle.

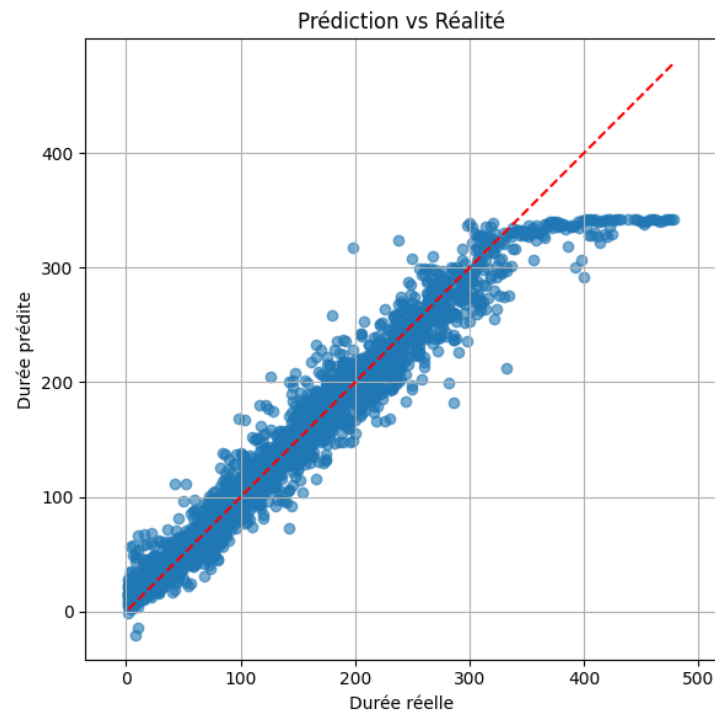


FIGURE 30 – Graphique de dispersion des données d'entraînement

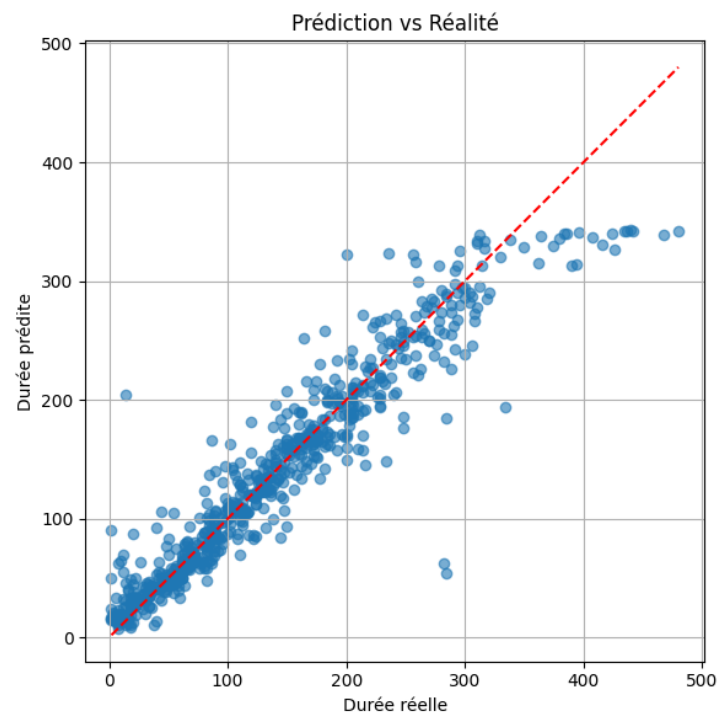


FIGURE 31 – Graphique de dispersion des données de test



6.2 Deuxième étape : Acquisition des signaux EMG

Pour effectuer l'acquisition des signaux électromyographiques (EMG) au niveau du biceps, j'ai utilisé l'écosystème MyoWare 2.0 développé par Advancer Technologies.¹ Le capteur [MyoWare 2.0 Muscle Sensor](#), placé sur le biceps de l'utilisateur à l'aide d'électrodes, capte les variations de potentiel électrique générées lors des contractions musculaires. Il délivre en sortie un signal analogique proportionnel à l'intensité de l'activité musculaire. Ce capteur est directement surmonté du [MyoWare 2.0 Wireless Shield](#), qui s'y connecte mécaniquement et électriquement via des broches pogo intégrées. Le shield se charge ensuite de convertir le signal analogique en données numériques à l'aide de son convertisseur analogique-numérique (ADC), puis de les transmettre en Bluetooth Low Energy (BLE) à un récepteur distant.

J'ai configuré ce module sur l'environnement Arduino à l'aide d'un modèle de script fourni dans la documentation officielle qui permet de lire en continu les valeurs du signal EMG avec une fréquence d'acquisition fixée à 1259 Hz, en cohérence avec la fréquence utilisée lors de la constitution de la base de données d'entraînement du modèle de prédiction de la fatigue. Le signal d'intérêt, est ensuite envoyé via BLE (Bluetooth low energy) vers un périphérique central.

Pour recevoir ces données, j'ai utilisé un smartphone équipé de l'application mobile gratuite Bluefruit Connect d'Adafruit. Celle-ci permet de visualiser les données EMG en temps réel sous forme textuelle ou graphique, ce qui a facilité le débogage et les premiers tests de réception BLE.

Une fois les données validées, j'ai utilisé un [SparkFun Thing Plus – ESP32 WROOM](#)², configuré comme client BLE. Je l'ai programmé pour détecter et se connecter automatiquement au module MyoWare Wireless Shield. Lorsqu'une connexion BLE est établie, l'ESP32 redirige les trames reçues vers son port série matériel (Serial1), que j'ai connecté directement à l'UART du STM32. Ainsi, le microcontrôleur embarqué reçoit les signaux EMG en temps réel, avec une latence minimale.

[Lien vers les scripts de configuration du MyoWare 2.0 Wireless Shield \(MyoWareBLEUART \(bouclier\)\) et du SparkFun Thing Plus – ESP32 WROOM \(MyoWareBLECentral\).](#)

1. Advancer Technologies est une petite entreprise américaine, spécialisée dans le développement de capteurs EMG destinés à enrichir les projets embarqués et académiques. Elle est connue pour avoir conçu le système MyoWare Muscle Sensor, et pour avoir récemment lancé l'écosystème modulaire MyoWare 2.0, composé de capteurs et shields compatibles Arduino.

2. Une petite carte de prototypage sans fil qui intègre un module Wi-Fi et Bluetooth (BLE), conçue pour simplifier la lecture et l'envoi de données entre des capteurs et un microcontrôleur.

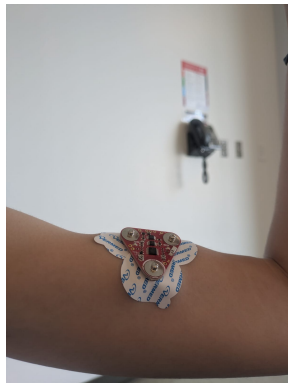


FIGURE 32 – Capteur MyoWare fixé sur le biceps

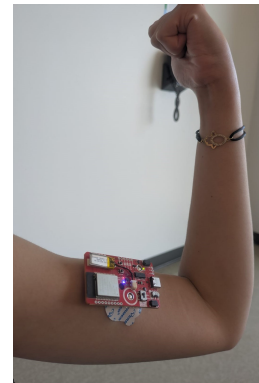


FIGURE 33 – Module Wireless Shield connecté au capteur



FIGURE 34 – Module SparkFun Thing Plus WROOM



FIGURE 35 – Module SparkFun installé sur l'ergocycle

6.3 Test du système

Le système embarqué fonctionne correctement sur le plan technique : les données EMG sont bien reçues par le microcontrôleur et traitées en temps réel par le modèle de prédiction intégré, pendant que l'ergocycle est en fonctionnement. Toutefois, un problème persiste : la prédiction du temps restant avant l'épuisement musculaire est constamment égale à zéro.

Deux hypothèses peuvent expliquer ce dysfonctionnement. La première concerne les features extraits du signal EMG. En effet, les certaines caractéristiques utilisées dans l'apprentissage du modèle sont liées à la forme du signal (RMS, MAV, ZC, WL). Or, le capteur que j'ai utilisé pour tester le système n'est pas identique à celui utilisé lors de la collecte initiale des données. Cette différence de capteurs pourrait avoir modifié la forme du signal, entraînant une baisse de performance du modèle.

La seconde hypothèse repose sur le type de mouvement réalisé. Le jeu de données d'apprentissage a été constitué à partir de mouvements de flexion-extension du coude, alors que les tests sur l'ergocycle impliquent un mouvement de pédalage. Or, les sollicitations musculaires

et les signaux EMG générés peuvent différer entre ces deux types d'efforts.

La solution à ces limitations consiste à effectuer une nouvelle collecte de données similaire à celle expliquée dans l'article que j'ai utilisé pour entraîner mon modèle³, mais cette fois ci directement sur l'ergocycle, en utilisant le capteur MyoWare 2.0 Wireless Shield, et en demandant à l'utilisateur de pédaler jusqu'à l'épuisement musculaire. Cette nouvelle base de données permettra de réentraîner le modèle dans des conditions réalistes et mieux adaptées à l'usage final.

7 Conclusion

Ce stage m'a permis de développer une version embarquée de l'ergocycle utilisé au sein du laboratoire S2M. Ce nouveau prototype fonctionne de manière autonome, sans ordinateur, et peut être utilisé à l'extérieur. J'ai pris en charge à la fois le développement logiciel incluant la logique de commande, la communication série, l'interface utilisateur et la gestion du multitâche avec FreeRTOS, ainsi que l'intégration matérielle : choix et montage des composants, gestion de l'alimentation, soudures, dépannage des courts-circuits et autres imprévus techniques.

Ce projet n'a pas été simple, mais j'ai réussi à surmonter les nombreux obstacles rencontrés. Le système final est stable, réactif et prêt à être utilisé comme support pour les travaux de recherche du laboratoire, notamment ceux portant sur la stimulation électrique fonctionnelle lors du pédalage. Il pourra également servir de base pour d'autres projets à venir.

7.1 Perspectives d'amélioration

Comme expliqué dans le rapport, la partie du projet liée à l'utilisation des signaux EMG pour prédire la fatigue reste à améliorer. Le modèle s'exécute correctement sur le microcontrôleur, mais pour le moment, il renvoie toujours une durée restante égale à zéro.

Pour corriger ça, il faudra organiser une collecte de données au sein du laboratoire S2M, avec le même capteur d'EMG utilisé dans mon projet et pendant un effort de pédalage jusqu'à fatigue. Ces données permettront de réentraîner le modèle correctement pour que le pilotage assisté par fatigue soit réellement opérationnel dans une version future de l'ergocycle.

3. *A Comprehensive Dataset of Surface Electromyography and Self-Perceived Fatigue Levels for Muscle Fatigue Analysis*



Table des figures

1	Architecture Hardware du projet	6
2	Contrôleur intégré au moteur	6
3	Contrôleur Flipsky FSESC 4.12 50A	7
4	Ecran Nextion intelligent de 4,3 pouces	8
5	Anti spark Switch Smart	8
6	Odrive Regen Clamp	10
7	Batterie Li-ion 48V – 10Ah	11
8	Vue arrière de l'ergocycle	11
9	Vue avant de l'ergocycle	11
10	Disposition des composants électriques sur l'ergocycle	12
11	Board Selector	14
12	Configuration du projet	15
13	Mode d'initialisation des périphériques par défaut	15
14	Configuration de la vitesse de Baud dans VESC Tool	16
15	Configuration de la vitesse de Baud dans Nextion Editor	16
16	Configuration du prescaler et de la formule de rechargement	17
17	Configuration de la fréquence de l'horloge LSI à 32KHz dans Clock Configuration	18
18	Architecture modulaire du code du projet	19
19	Configuration des seuils de fonctionnement du contrôleur sur VESC Tool	20
20	Etape 1 : usage	21
21	Etape 2 : Motor	21
22	Etape 3 : Battery	22
23	Etape 4 : Setup	22
24	Etape 4 : Setup	23
25	Interface de NetionEditor	24
26	Interface utilisateur : pages 1	25
27	Interface utilisateur : pages 3 et 4	26
28	Création de la tâche TaskUpdateFromS	29
29	Création de la tâche TaskUpdateS	29
30	Graphique de dispersion des données d'entraînement	34
31	Graphique de dispersion des données de test	34
32	Capteur MyoWare fixé sur le biceps	36
33	Module Wireless Shield connecté au capteur	36
34	Module SparkFun Thing Plus WROOM	36
35	Module SparkFun installé sur l'ergocycle	36